



Criptografía y Seguridad

Criptografía:
Cifrado

Criptosistema (repaso)

Es una terna de algoritmos

Gen: $() \rightarrow K$

Generador de clave

Enc: $K \times P \rightarrow C$

Cifrado

Dec: $K \times C \rightarrow P$

Descifrado

Propiedad básica:

Para todo m y k válidos: $d_k(e_k(m))=m$

Conjuntos involucrados:

K , espacio de claves es el conjunto de todas las claves posibles

P , espacio plano, es el conjunto de todos los mensajes posibles

C , espacio cifrado, es el conjunto de todos los mensajes cifrados posibles

Secreto perfecto (repaso)

Dado un criptosistema: Gen, e y d

El criptosistema posee secreto perfecto si para toda distribución de probabilidades en M , cada mensaje m y cada mensaje cifrado c tal que $\Pr[C = c] > 0$:

$$\Pr[M=m \mid C = c] = \Pr[M=m]$$

La paradoja del secreto perfecto:

Notar que $c=e_k(m)$, así que las variables aleatorias discretas C y E son dependientes. Sin embargo, la propiedad de secreto perfecto interpretada probabilísticamente dice que son V.A.D.s independientes.

One Time Pad

Atribuido a Verman (1917)

Gen: $k \leftarrow \{0, 1\}^n$, $n=|m|$

Enc: $e_k(m) = m \oplus k$

Dec: $d_k(c) = c \oplus k$

Ejemplo:

m=	0	0	1	0	1	1	0	1	0	0	0	1	0	1	1	1	0
k=	0	1	1	0	0	1	1	1	0	1	0	0	1	1	0	1	0
c=	0	1	0	0	1	0	1	0	0	1	0	1	1	0	1	0	0

Propiedad fundamental del OTP



El OTP tiene secreto perfecto

Demostración (1)

¿El OTP tiene secreto perfecto?

Para cualquier par (m, c) de longitud n , $|k|=2^n=N$

Queremos probar: $P(M=m|C=c)=P(M=m)$

Lema 1:

$$P(C=c) = \text{Sum}_k(P(C=c \cap K=k))$$

k y m se eligen
independientemente

$$\begin{aligned} P(C = c \cap K = k) &= P(M = c \oplus k \cap K = k) \\ &= P(M = c \oplus k) * P(K = k) \\ &= P(M = c \oplus k) * (1/N). \end{aligned}$$

$$P(C=c) = (1/N) * \text{Sum}_k(P(M = c \oplus k))$$

$$P(C=c) = (1/N)$$

Pasamos por todos los
valores de m . La suma de las
probabilidades da 1

Demostración (2)

¿El OTP tiene secreto perfecto?

Para cualquier par (m, c) de longitud n , $|k|=2^n=N$

$$\begin{aligned} P(M = m \mid C = c) * P(C = c) &= P(C = c \cap M = m) \\ &= P(K = c \oplus m \cap M = m) \\ &= P(K = c \oplus m) * P(M = m) \end{aligned}$$

$$P(M = m \mid C = c) * (1/N) = (1/N) * P(M = m)$$

$$P(M = m \mid C = c) = P(M = m)$$

One Time Pad

Las malas noticias

Secreto perfecto $\Rightarrow |K| \geq |C|$

Si una clave se utiliza 2 veces:

$$c_1 = m_1 \oplus k$$

$$c_2 = m_2 \oplus k$$

$$\Rightarrow c_1 \oplus c_2 = m_1 \oplus m_2$$

Se pierde el secreto perfecto

La clave seleccionada **DEBE** ser aleatoria

¿Cómo puede garantizarse esto?

One Time Pad

¿Por qué la clave debe ser aleatoria?



Ejercicio

Considerar una clave no aleatoria:

$$P(k=00) = 0.3 \text{ y } P(k=01) = 0.1$$

$$P(k=10) = 0.4 \text{ y } P(k=11) = 0.2$$

Si la distribución del texto plano es:

$$P(m=00) = 0.60 \quad P(m=01) = 0.15$$

$$P(m=10) = 0.10 \quad P(m=11) = 0.15$$

Y se obtiene un mensaje $C=01$

¿Que probabilidades hay que $M=00$)

Ejercicio

Queremos calcular $P(m=00 \mid c=01)$.

Estadísticamente M y C no son independientes:

$$P(M=x \mid C=y) * P(C=y) = P(C=y \mid M=x) * P(M=x)$$

$$P(M=00 \mid C=01) * \mathbf{P(C=01)} = \mathbf{P(C=01 \mid M=00)} * P(M=00)$$

$$\mathbf{P(C=01)} = \text{Sum}_k(P(C=01 \cap K=k))$$

$$= \text{Sum}_k(P(M=01 \oplus k) * P(K=k))$$

$$= P(M=01) * P(K=00) + P(M=00) * P(K=01) +$$

$$P(M=11) * P(K=10) + P(M=10) * P(K=11)$$

$$= 0.15 * 0.3 + 0.6 * 0.1 + 0.15 * 0.4 + 0.1 * 0.2$$

$$= \mathbf{0.185}$$

Ejercicio

$$P(C=01 \mid M=00) = P(K=01 * M \mid M=00) = P(K=01) = 0.1$$

Juntando todo:

$$P(M=00 \mid C=01) * P(C=01) = P(C=01 \mid M=00) * P(M=00)$$

$$P(M=00 \mid C=01) * 0.185 = 0.1 * 0.6$$

$$P(M=00 \mid C=01) = 0.32$$

$$P(M=00 \mid C=01) = 0.32 \quad \neq \quad P(M=00) = 0.6$$

Más allá del OTP

Resultado teóricos interesantes

Cualquier criptosistema con secreto perfecto es reducible al OTP

Cualquier sistema que no sea reducible al OTP no posee secreto perfecto

Consecuencias

El secreto perfecto es demasiado impráctico

Necesitamos otras construcciones

Seguridad Computacional

Secreto perfecto = Seguridad incondicional



Seguridad Computacional

Secreto perfecto = Seguridad incondicional



- Limitar escenarios
- Limitar garantías

Seguridad computacional

Seguridad Computacional



Garantizar seguridad
solo contra
adversarios “limitados”

Asume un límite en los
recursos del atacante
(especialmente tiempo).

Aceptar una pequeña
probabilidad de éxito
para el atacante

Se deja de lado la
infalibilidad

Criptosistemas de flujo

Intercambian la clave del OTP por la salida de un generador pseudoaleatorio:

Gen: $k \leftarrow K$

Enc: $e_k(m) = G(k) \oplus m$

Dec: $d_k(c) = G(k) \oplus c$

La gran diferencia: $|K| \lll |M|$

Por ejemplo: $|K| = 2^{128}$, $|M| = |K|^{128}$

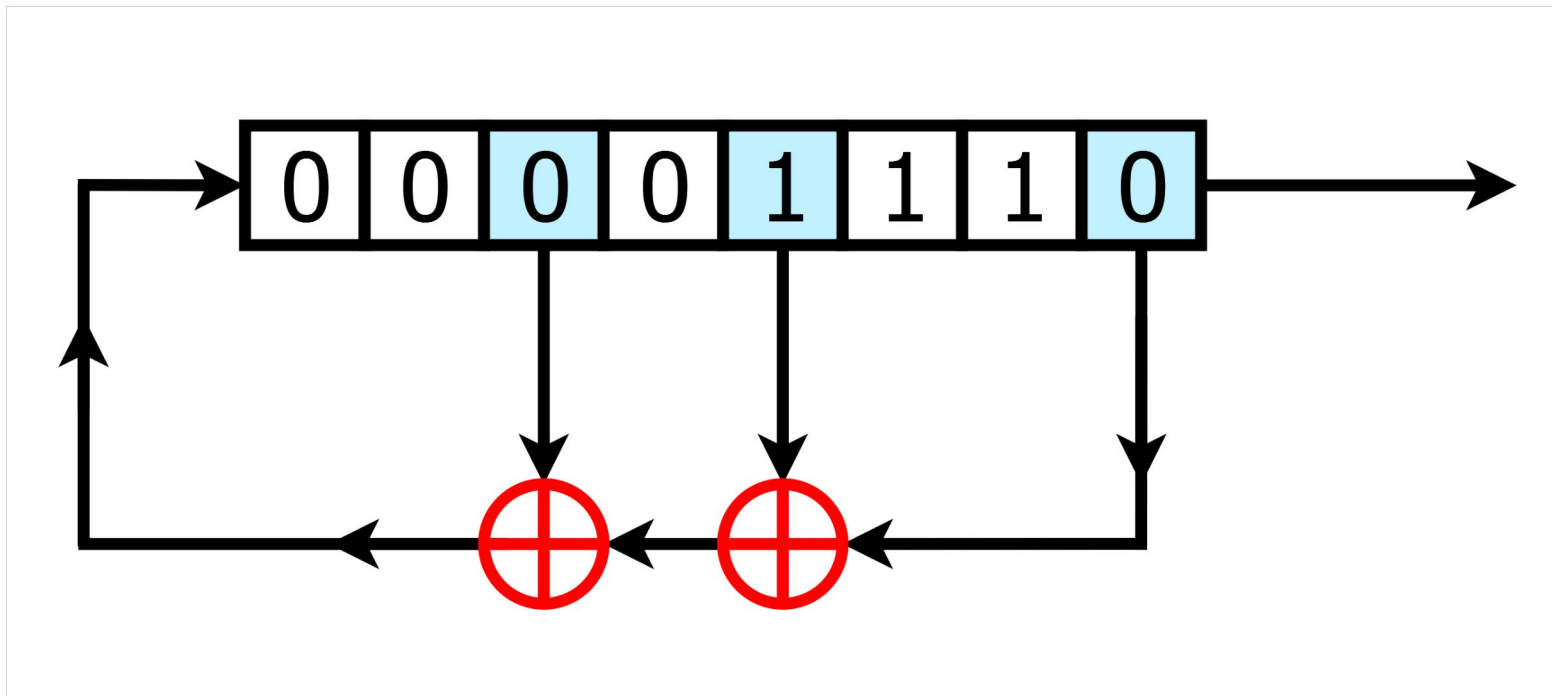
Generadores pseudoaleatorios

Son algoritmos determinísticos

Expanden una entrada llamada semilla (*seed*)

La salida parece aleatoria

Ejemplo:



Generadores pseudoaleatorios

Formalmente:

Sea $D = \{ f: \{0,1\}^n \rightarrow \{0,1\} \}$ una familia de funciones

$G: \{0,1\}^s \rightarrow \{0,1\}^n$, con $s < n$ es un generador pseudoaleatorio con respecto a D si:

Para toda f en D :

$$P(f(G(r^s)) \neq f(r^n)) = \epsilon$$

Probabilidad
generalizada

Secuencia realmente
aleatoria

Valor despreciable

Ejemplo

Sea $s = \{ 1, \dots, 10 \}$

$G(s) = \{ G_0 \% 2, G_1 \% 2, G_2 \% 2, \dots, G_n \% 2 \}$

$$G_0 = s$$

$$G_i = G_{i-1} * 3 + 1 \bmod 11$$

Generar 5 bits con $s = 2$

Generar 5 bits con $s = 6$

Criptosistemas de flujo

Intercambian la clave del OTP por la salida de un generador pseudoaleatorio:

Gen: $k \leftarrow K$

Enc: $e_k(m) = G(k) \oplus m$

Dec: $d_k(c) = G(k) \oplus c$

¿Cómo medimos la seguridad del criptosistema ?

Pruebas de seguridad

Prueban características de un criptosistema

Son una serie de pasos que prueban un algoritmo (que representa un ataque)

El atacante gana o pierde la prueba

Se puede repetir múltiples veces

Interesa la probabilidad de éxito del atacante

Prueba de indistinguibilidad

Eavesdropping Indistinguishability test: $\text{Eav}_{A,\Pi}$

Dado un adversario A , y un Criptosistema Π :

- 1) A genera m_0 y m_1 arbitrariamente
- 2) Se genera una clave $k \leftarrow K$
- 3) Se genera $b \leftarrow \{0, 1\}$
- 4) A recibe $c = e_k(m_b)$
- 5) A emite $b' = \{0, 1\}$

$\text{Eav}_{A,\Pi} = 1$ si $b = b'$ (A gana)

Si $\Pr[\text{Eav}_{A,\Pi} = 1] = 0,5 + \varepsilon \Rightarrow \Pi$ es indisting.

Nivel de seguridad

Nivel de seguridad

Está dado por una variable

Relaciona la cota en el poder de un adversario y la probabilidad de éxito que tendrá

Dado un nivel de seguridad n se espera que:

Un adversario corra algoritmos de orden PPT(n)
(Probabilistic Polynomial Time)

La probabilidad de éxito sea una función despreciable en n :

$$\varepsilon(n) \text{ es despreciable} \leftrightarrow \lim \varepsilon(n) < 1 / n^k$$

Prueba de indistinguibilidad

Eavesdropping Indistinguishability test: $\text{Eav}_{A,\Pi}$

Dado un adversario $A(n)$, y un Criptosistema $\Pi(n)$:

- 1) A genera m_0 y m_1 arbitrariamente
- 2) Se genera una clave $k \leftarrow K$
- 3) Se genera $b \leftarrow \{0, 1\}$
- 4) A recibe $c = e_k(m_b)$
- 5) A emite $b' = \{0, 1\}$

$\text{Eav}_{A,\Pi} = 1$ si $b = b'$ (A gana)

Si $\Pr[\text{Eav}_{A,\Pi} = 1] = 0,5 + \varepsilon(n) \Rightarrow \Pi$ es indisting.

Criptosistemas de flujo

Intercambian la clave del OTP por la salida de un generador pseudoaleatorio:

$$\text{Gen: } k \leftarrow K$$

$$\text{Enc: } \text{enc}_k(m) = G(k) \oplus m$$

$$\text{Dec: } \text{dec}_k(c) = G(k) \oplus c$$

Teorema:

Si $G(\cdot)$ es un generador pseudoaleatorio entonces el criptosistema es indistinguible ante observadores

Ejercicio

Demostrar que si es posible distinguir $G(\cdot)$ de una secuencia aleatoria un criptosistema de flujo basado en G no pasa la prueba EAV

Múltiples cifrados

Multiple message eavesdropping test: $\text{Mul}_{A,\Pi}$

Dado un adversario A , y un Criptosistema Π :

- 1) A genera $(m_{00}, m_{01}, \dots, m_{0i})$ y $(m_{10}, m_{11}, \dots, m_{1i})$
- 2) Se genera una clave $k \leftarrow K$
- 3) Se genera $b \leftarrow \{0, 1\}$
- 4) A recibe $(c_0, c_1, \dots, c_i)$, donde $c_j = \text{Enc}_k(m_{bj})$
- 5) A emite $b' = \{0, 1\}$

$\text{Mul}_{A,\Pi} = 1$ si $b = b'$ (A gana)

Si $\Pr[\text{Mul}_{A,\Pi} = 1] = 0.5 + \varepsilon \Rightarrow \Pi$ es indisting.

Ejercicio

Los criptosistemas de flujo NO son seguros bajo múltiples cifrados

$$c_1 = m_1 \oplus G(s)$$

$$c_2 = m_2 \oplus G(s)$$

$$\Rightarrow c_1 \oplus c_2 = m_1 \oplus m_2$$

¿Similar al problema del One Time Pad con reuso de claves?
¡No es casualidad!

Definir un ataque que gane la prueba MUL

Solución

Definir un Algoritmo A que gane la prueba de múltiples cifrados

$A \rightarrow (m_{00}=0\dots 0, m_{01}=0\dots 0), (m_{10}=0\dots 0, m_{11}=1\dots 1)$

A obtiene c_1, c_2

$$X = c_1 \oplus c_2 = m_1 \oplus m_2$$

Si $x = 0\dots 0 \rightarrow b=0$

Si no $\rightarrow b=1$

Necesidad de cifrado probabilístico

Si una función de cifrado es determinística, NO es segura bajo múltiples cifrados

El adversario anterior aplica a cualquier criptosistema donde $e_k(x)$ es constante



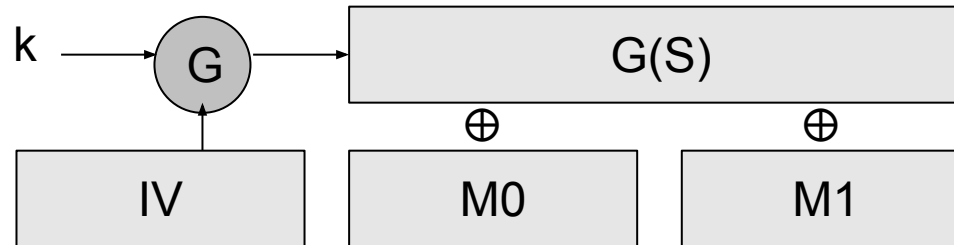
Evitar reutilizar la clave

Se agrega un valor a la función generadora

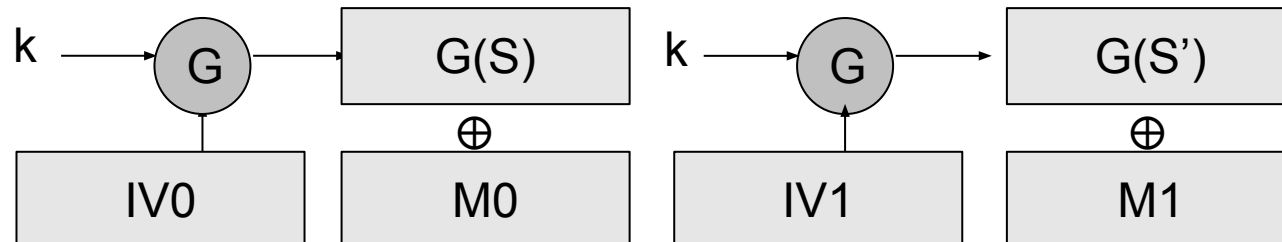
Dicho valor no deberá repetirse para una misma clave (se lo llama nonce o IV)

Dos formas:

Modo sincronizado (un unico IV)



Modo no sincronizado (un IV por mensaje)



Ataques de texto plano escogido

Chosen Plain Text indistinguishability: $\text{CPA}_{A,\Pi}$

Dado un adversario A , y un Criptosistema Π :

- 1) Se genera una clave $k \leftarrow K$
- 2) A obtiene $f(x)=e_k(x)$ y genera (m_0, m_1)
- 3) Se genera $b \leftarrow \{0, 1\}$
- 4) A recibe $c = e_k(m_b)$
- 5) A emite $b' = \{0, 1\}$

$\text{CPA}_{A,\Pi} = 1$ si $b = b'$ (A gana)

Si $\Pr[\text{CPA}_{A,\Pi}=1] = 0.5 + \varepsilon \Rightarrow \Pi$ es indisting.

Propiedades CPA

Un criptosistema determinístico no puede ser CPA-Secure

Si un criptosistema es CPA-Secure para un mensaje también lo es para múltiples

Un criptosistema que es CPA-Secure, pero de tamaño limitado (cifra mensajes de hasta n bits) puede ser extendido arbitrariamente

$$m = m_0 || m_1 || \dots || m_i \quad (|m_j| = n \text{ bits})$$

$$\text{enc}_k(m) = \text{enc}_k(m_0) || \text{enc}_k(m_1) || \dots \text{enc}_k(m_i)$$

Primitivas de cifrado en bloque

Están definidos para mensajes de tamaño FIJO

$$K = \{0, 1\}^n, \quad C=P= \{0, 1\}^b$$

$$\text{Gen: } k \leftarrow K$$

$$\text{Enc: } e_k(m)=c$$

$$\text{Dec: } d_k(c)=m$$

Son funciones pseudoaleatorias

Podrían “usarse” como criptosistemas, pero son determinísticas → No pasan la prueba Mul

Se combinan con mecanismos de encadenamiento para formar criptosistemas seguros.

Extensión y encadenamiento

¿Que ocurre el mensaje a cifrar es más chico que el tamaño de bloque?

Se lo extiende sistemáticamente:

Simple Pad:

- Completar con ceros

- Es necesario conocer el tamaño real del mensaje

Des Pad:

- Agregar un bit 1 y luego bits en 0

- Puede agregar un bloque completo de padding

Extensión y encadenamiento

¿Que ocurre el mensaje a cifrar es más grande que el tamaño de bloque?

- Se divide el mensaje en bloques

- Se extiende el ultimo bloque

- Se transforma cada bloque según algún modo de encadenamiento

Modos de encadenamiento

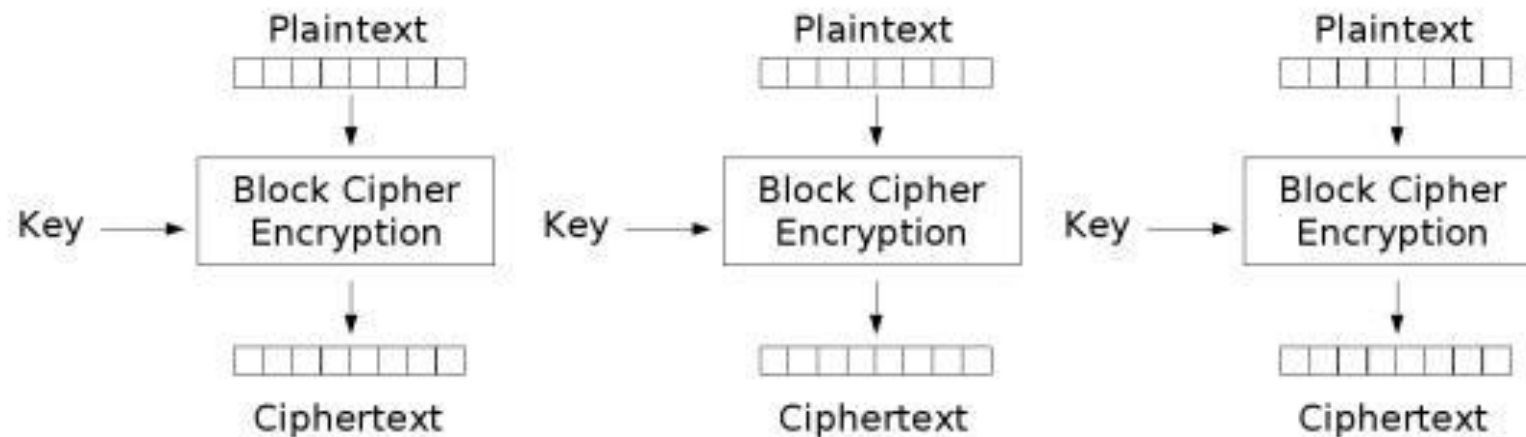
- Objetivo → extender una primitiva de cifrado a bloques mayores a su tamaño

- Hay diversos modos con diferentes propiedades

- No todos son aplicables a cada problema

Encadenamiento ECB

Trata al mensaje original como un conjunto de bloques independientes



Electronic Codebook (ECB) mode encryption

NO es CPA-Secure
(NO UTILIZAR)

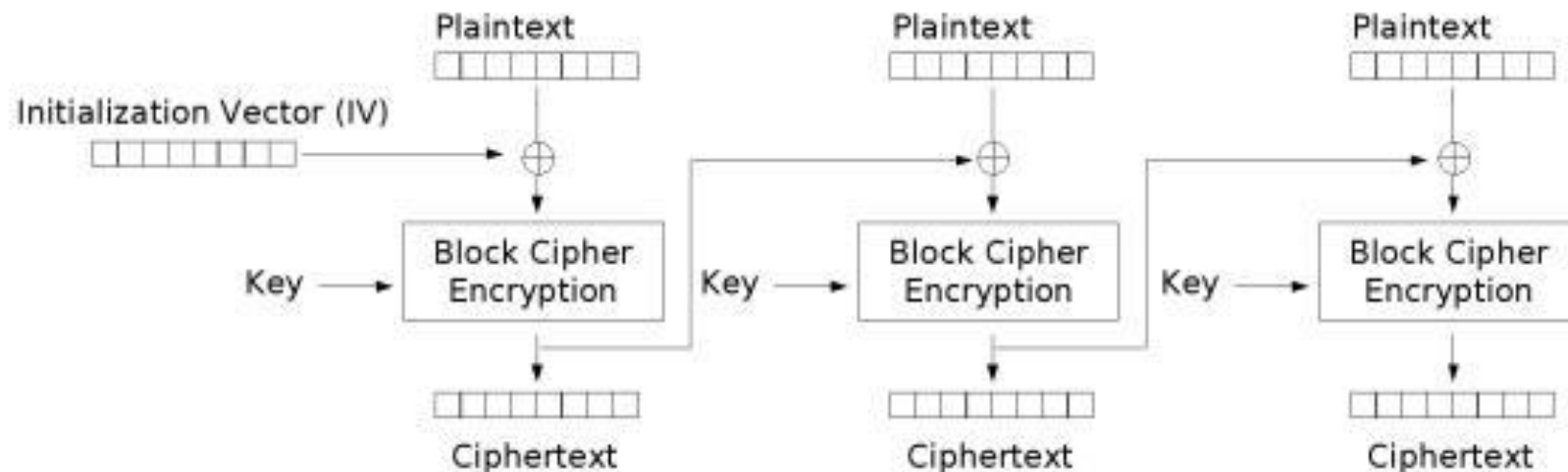


Encadenamiento CBC

Utiliza la salida de un bloque como entrada para el próximo

Disminuye el traspaso de información

Requiere un valor inicial (IV) ALEATORIO



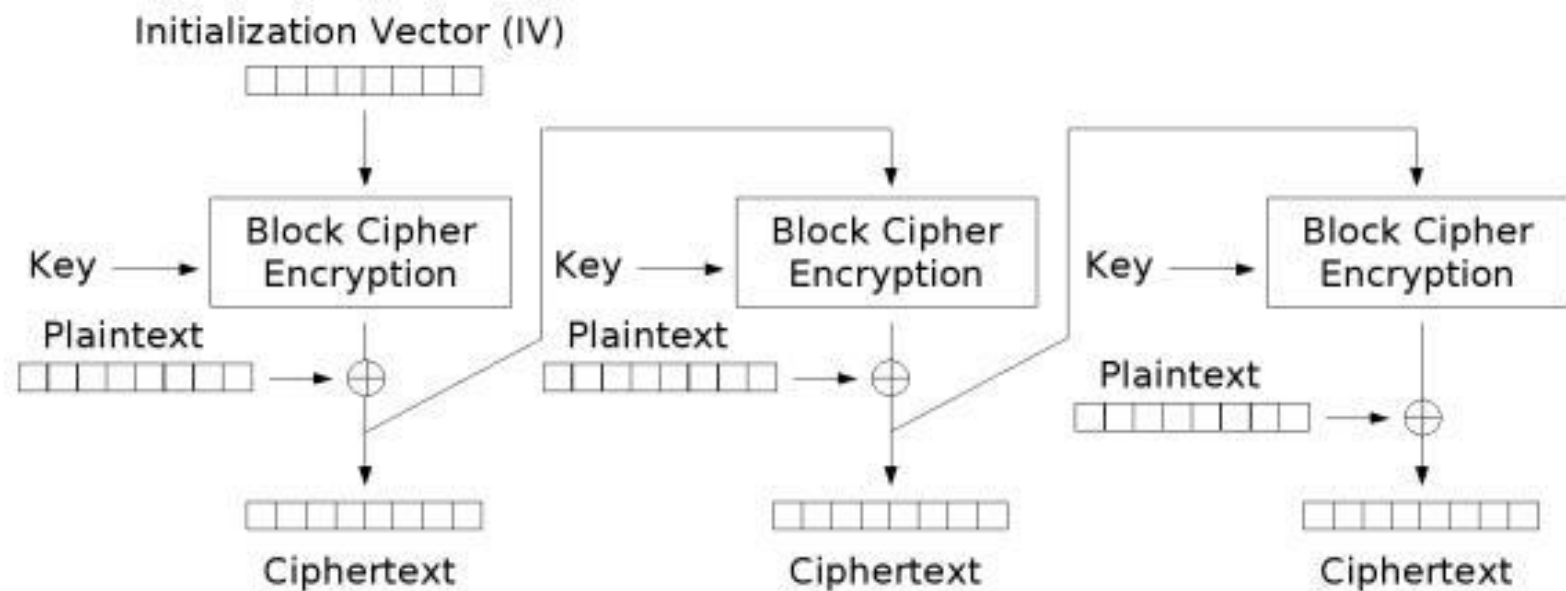
Cipher Block Chaining (CBC) mode encryption

Encadenamiento CFB

Utiliza solo la función Enc

Menor complejidad para dispositivos embebidos

Permite generar una transformación de flujo a partir de una de bloque

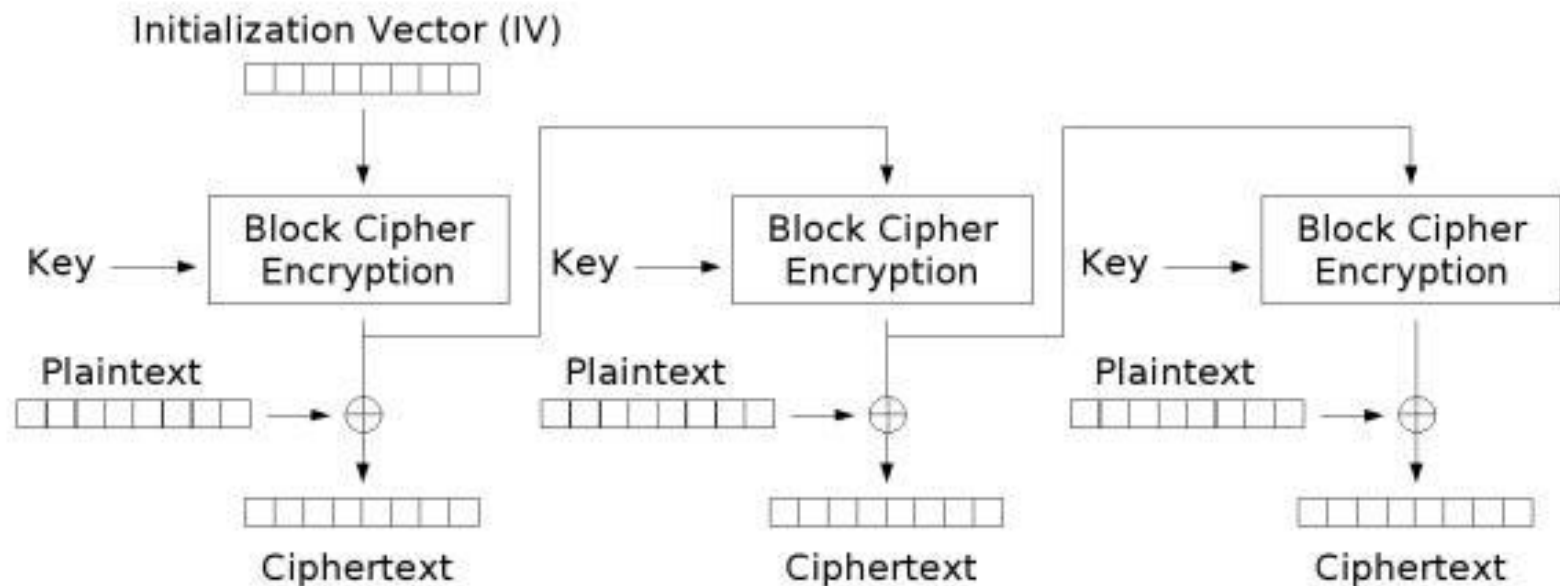


Cipher Feedback (CFB) mode encryption

Encadenamiento OFB

Construye una transformación de flujo a partir de una de bloque

Permite calcular los bits de la transformación por adelantado



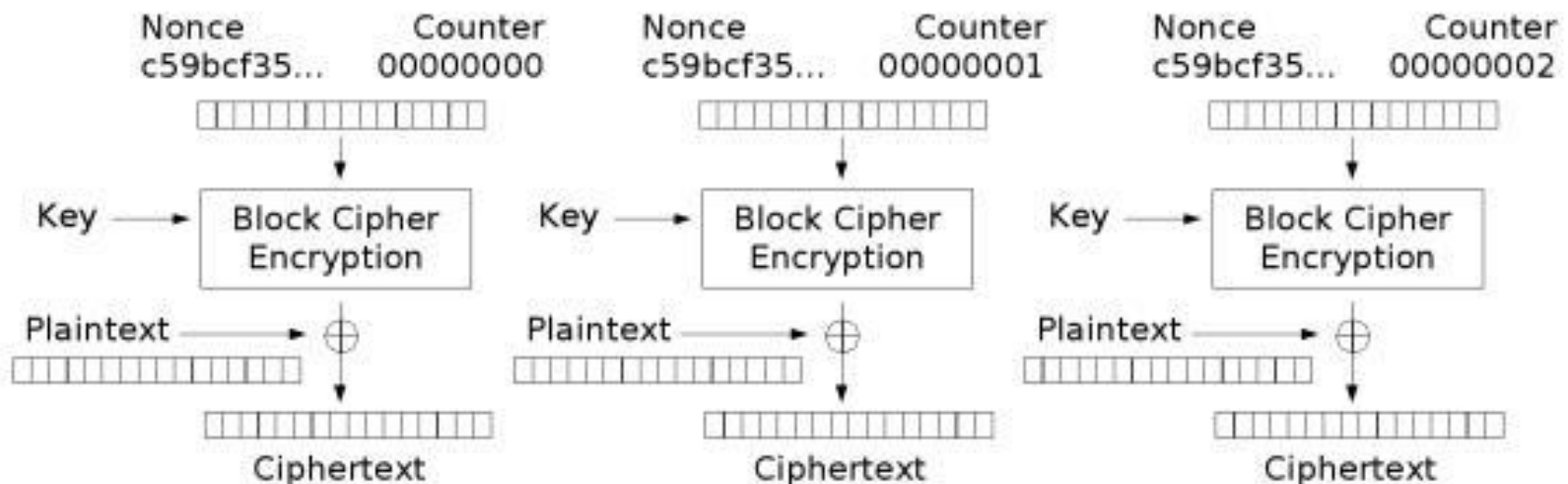
Output Feedback (OFB) mode encryption

Encadenamiento Counter

Utiliza un contador para generar secuencias de bits de clave

Permite acceso aleatorio

Muy útil en ambientes con capacidad de procesamiento paralelo



Counter (CTR) mode encryption

Seguridad de cifrado por bloques

Las pruebas son por reducción a propiedades de la primitiva subyacente

Requieren que la primitiva se comporte como una función pseudoaleatoria

Esto significa que no es posible distinguir la función $f(x) = \text{Enc}_k(x)$ de una función tomada al azar del conjunto de funciones del mismo dominio.

Dicho de otra manera, significa que para cada posible k , $f(x) = \text{Enc}_k(x)$ es un generador pseudoaleatorio

Seguridad de cifrado por bloques

Si la primitiva es una función pseudoaleatoria

CBC es CPA-Secure con IVs aleatorios

OFB, CFB son CPA-Secure con IVs aleatorios

Counter es CPA-Secure si no se repite (k, nonce)



No está demostrado que existan
las funciones pseudoaleatorias

DES – Data Encryption Standard

Método desarrollado por IBM

Adoptado por el gobierno de EE.UU. como estándar para usos no militares

Fue la primera función de cifrado pública apoyada por un gobierno

Alcanzó uso comercial masivo

Características

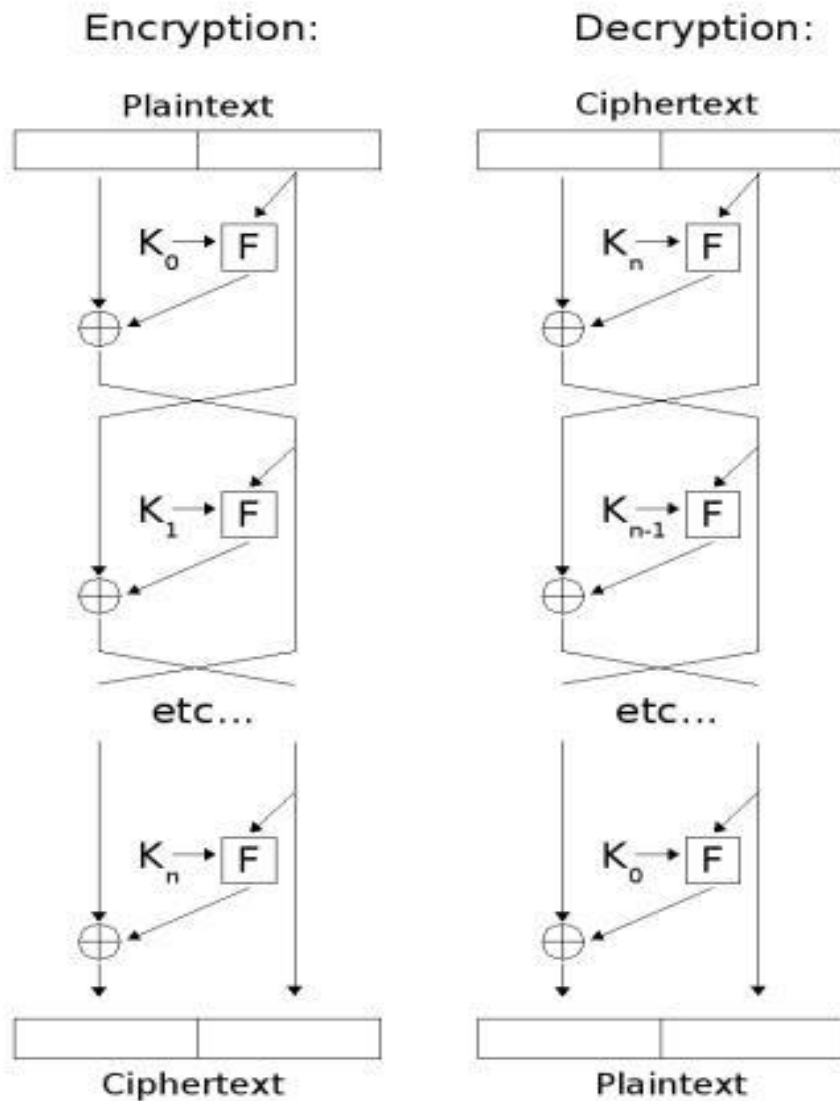
Trabaja con textos planos binarios

Entrada: 64 bits

Clave: 64 bits (56 bits efectivos)

Salida: 64 bits

DES – Data Encryption Standard



Feistel Cipher

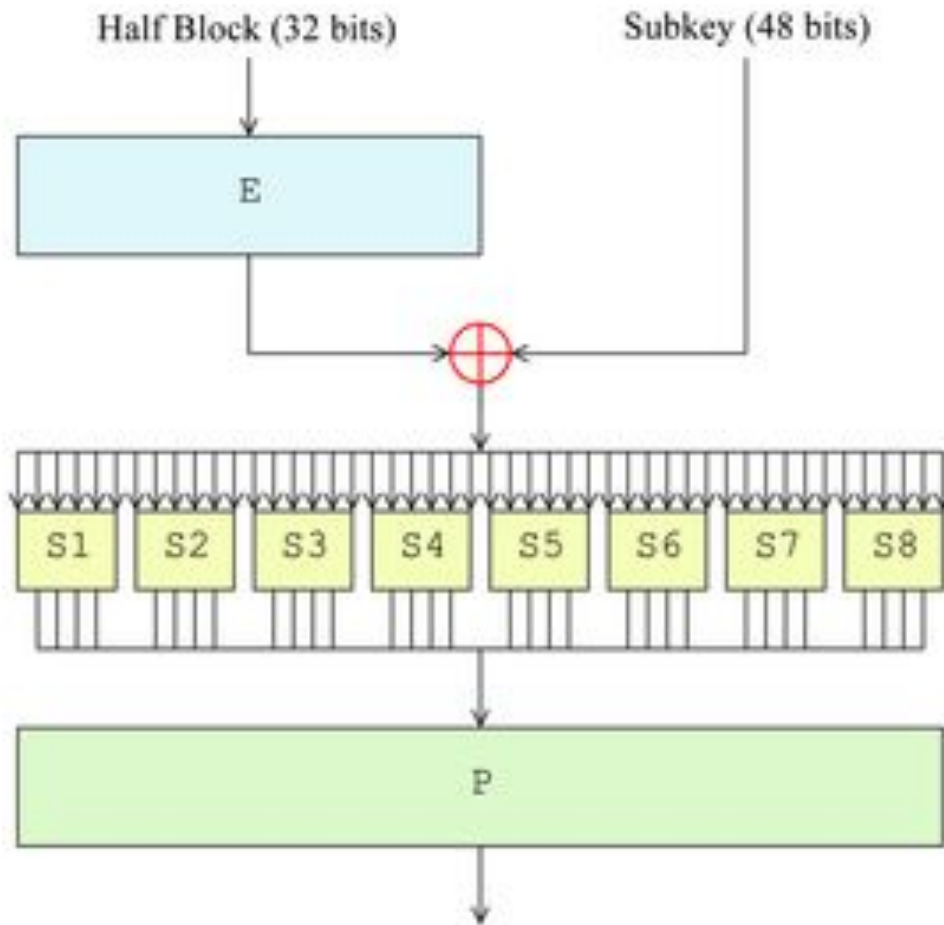
Se parte el mensaje en dos mitades

Se transforma una mitad con parte de la clave

Se intercambian las dos mitades de lugar

Se repiten los tres pasos anteriores (una ronda) 16 veces

La función de transformación



$E \rightarrow$ Expansión

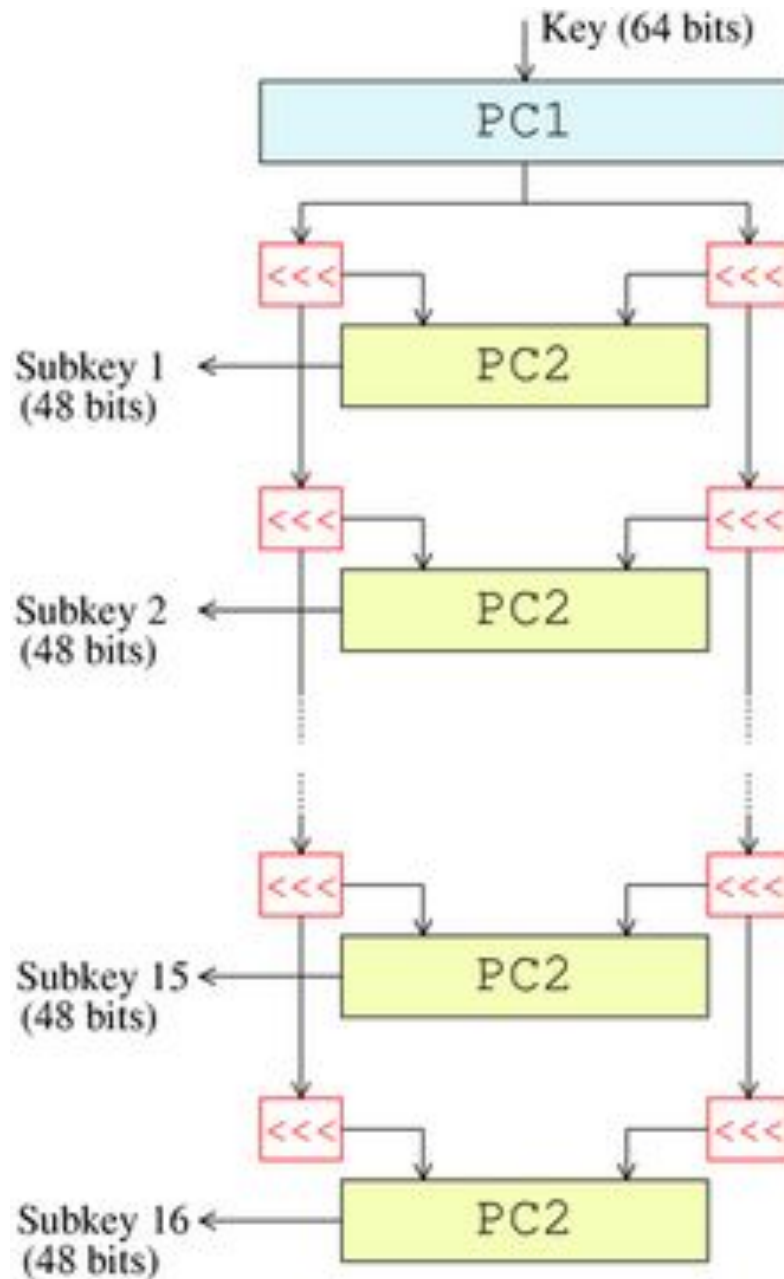
32 a 48 bits

$S_x \rightarrow$ Sustituciones

6 a 4 bits

$P \rightarrow$ Permutación

Generación de subclaves



PC1 → Selección de 56 bits
8 restantes son paridad

PC2 → separación en dos mitades

Generación de subclave
24 bits de cada mitad

<<< → despl. a nivel bits

Evolución

Nivel teórico de seguridad: 56 bits

Fuerza bruta: probar 2^{56} claves

Generó desconfianza debido a cuestiones de diseño confidenciales (sustituciones)

1990 - Criptoanálisis diferencial

Permite atacar a DES en 2^{47} intentos

Gran parte del diseño de DES disminuye el impacto de este ataque (otros sistemas fueron inmediatamente quebrados)

1992 – Criptoanálisis lineal

Permite atacar a DES en 2^{43} intentos

3-DES

Variante fuerte de DES

Consisten en seleccionar 3 claves independientes y calcular

$$c = \text{Enc}_{k_1}(\text{Dec}_{k_2}(\text{Enc}_{k_3}(p)))$$

Brinda una seguridad del orden de 112 bits

Notar que es menor a 168 bits, debido a un ataque conocido como meet-in-the middle

Es inmune a criptoanálisis diferencial y lineal

Es mucho más lenta que des (3 veces!)

AES

Reemplazo de DES

Fue seleccionado mediante un concurso internacional abierto (5 años)

Orientado a bloques de 128 bits

Clave de 128, 192 o 256 bits

Es la primitiva de cifrado **recomendada** en la actualidad

Esquema de funcionamiento

4 etapas por cada ronda

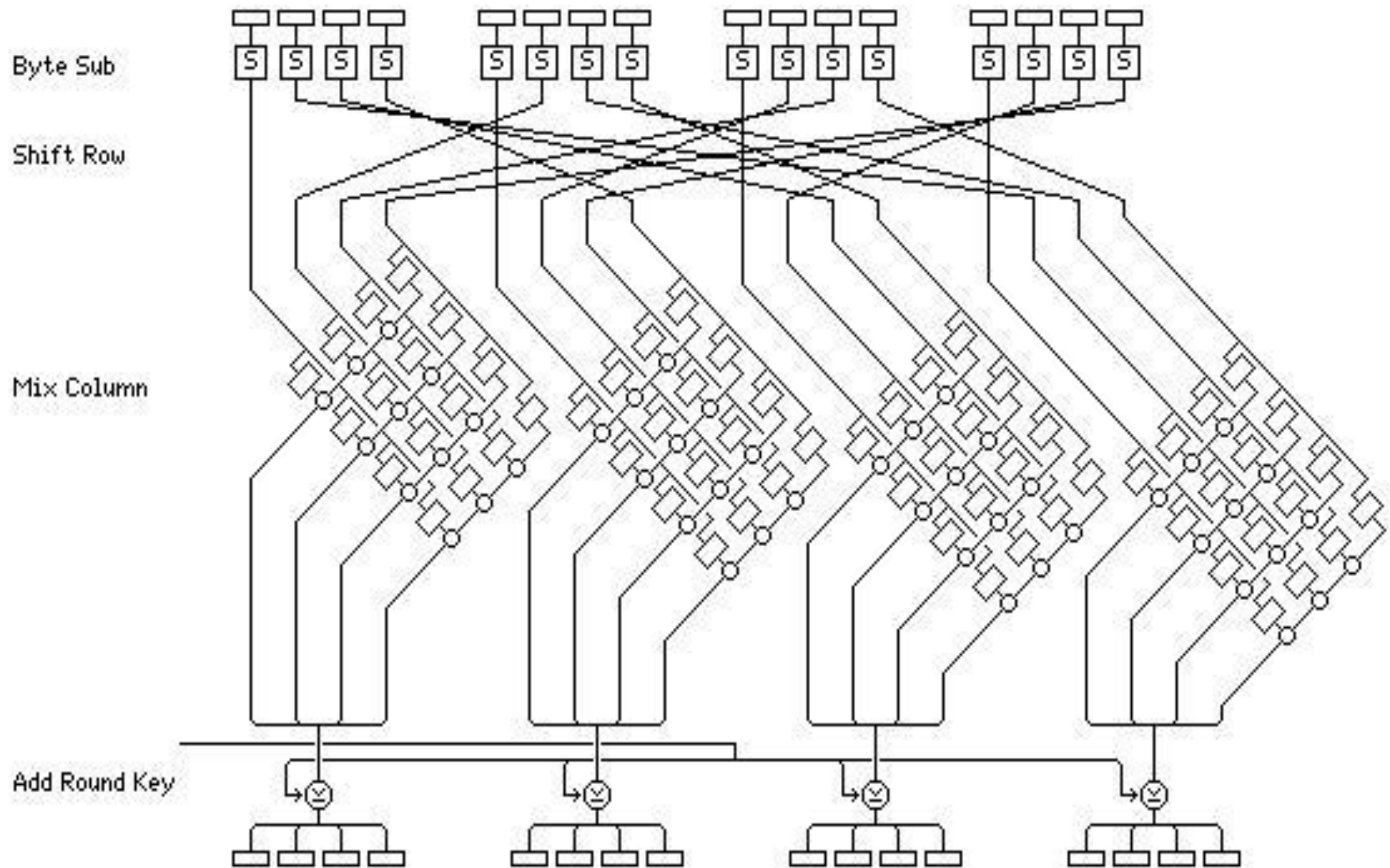
Byte Sub: Sustitución de valores según una tabla derivada de invertir una matriz en campo $GF(2^8)$

Shift Row: Permutación de bits

Mix Column: transformación lineal invertible de cada byte en función de los 4 que forman el mismo grupo

Add round key: Xor con la parte de la clave derivada para la ronda en cuestión

Esquema de funcionamiento



Esquema de funcionamiento

Variaciones para generar asimetrías

1ra Ronda

Solo tiene el paso Add Round Key

Última ronda

No tiene paso Mix Column

Round keys (128 bits)

Los primeros 16 bytes:

corresponden a la clave original

16 bytes más:

Generar máscara

Tomar los últimos 4 bytes y rotarlos 8 bits a la derecha

Aplicar ByteSub a cada byte

Al byte menos significativo: xor con 2^i , siendo i el número de grupo de bytes a generar (1 los primeros 16)

Generar grupos de 4 bytes

1^{ros} 4 bytes: mascara xor key bytes generados 16 posiciones antes

Siguientes: xor entre los 4 bytes generados y los generados 16 posiciones antes

Criptosistemas en proyectos

Se considera mala práctica desarrollar un criptosistema nuevo para un proyecto

Existen funciones que han sido estudiadas por años para llegar a un nivel de confianza adecuado

Continuamente se publican avances que afectan la seguridad de funciones existentes

¿Cómo elegir?

Estado de un criptosistema

Criptosistema seguro

Cumple con las expectativas de su modelo de seguridad



Estado de un criptosistema

Criptosistema debilitado

Existen adversarios con probabilidades no despreciables de éxito

Pero el esfuerzo es muy alto (ej. décadas) o las condiciones muy difíciles (ej. disponer de 2^{80} mensajes)



Estado de un criptosistema

Criptosistema quebrado

Existen adversarios con probabilidades no despreciables de éxito en tiempos practicables



Escenarios

Un criptosistema puede ser seguro y estar quebrado al mismo tiempo

Hay múltiples pruebas de seguridad

Cada una prueba un escenario diferente



Criptosistemas de flujo recomendados

Funciones $G(\cdot)$:

~~RC4 (usado en https y WEP)~~

~~CSS (usado en DVDs)~~

~~A5/1, A5/2 (GSM)~~

~~E0 (Bluetooth)~~

Salsa20: $\{0,1\}^{128 \text{ o } 256} \times \{0,1\}^{64} \rightarrow \{0,1\}^n, n=2^{64} \cdot 2^9$

Rabbit: $\{0,1\}^{128} \times \{0,1\}^{64} \rightarrow \{0,1\}^n, n=2^{128}$

IV

Criptosistemas de bloque recomendados

~~DES: $\{0,1\}^{56} \times \{0,1\}^{64} \rightarrow \{0,1\}^{64}$~~

IDEA: $\{0,1\}^{128} \times \{0,1\}^{64} \rightarrow \{0,1\}^{64}$

IDEA-CBC, IDEA-CTR

3DES: $\{0,1\}^{112 \text{ o } 168} \times \{0,1\}^{64} \rightarrow \{0,1\}^{64}$

3DES-CBC, 3DES-CTR

Recomendado para
proyectos nuevos

AES: $\{0,1\}^{128,192 \text{ o } 256} \times \{0,1\}^{128} \rightarrow \{0,1\}^{128}$

AES-CBC, AES-CTR

Cifrado de bloque - tamaños

Espacios de clave:

> 64 bits. AES tiene 128, 192 o 256 bits

Espacio de mensajes:

≥ 128 bits

Para comparar:

Partículas en el universo: $\sim 2^{88}$ átomos.

Edad estimada del universo: $\sim 2^{58}$ seg.

Lectura Recomendada

Capítulos 2 y 3

Introduction to Modern Cryptography
Katz & Lindell
